

SILENT SHIFT

Team Work Distribution & Per-Person Technical Specification

3-PERSON BUILD PLAN — v1

MIT Hackathon · Problem Statement 16 · Peace, Justice and Strong Institutions

Prepared by Mohammed Shamaz

September 12, 2026

Contents

1. Overview & Strategy
2. Shared Contracts
3. Person 1 — Frontend — Complete Specification
4. Person 2 — Backend: Data & Detection — Complete Specification
5. Person 3 — Backend: Fusion, Drift & API — Complete Specification
6. Master Build Timeline
7. Git & Merge-Conflict Prevention
8. Sync Points & Handoff Checklist

1. Overview & Strategy

The team splits into three ownership zones — one frontend, two backend — divided along the exact seam where the system architecture already has a natural data handoff. Each person works inside their own folder for the entire build and never edits a file another person owns, which removes git merge conflicts by construction rather than by discipline.

The dependency problem is solved the same way: every boundary between two people is a written contract, agreed once in the first hour, that both sides build against from minute one — using mocked data on either side of the contract until the real thing is ready. Nobody sits idle waiting on a teammate, and nobody edits a shared file to stay in sync.

Ownership at a glance

Person	Owns	Folders
Person 1	Frontend — dashboard, queue, case view, feedback control	/frontend
Person 2	Backend — data ingestion, cohort construction, self-baseline and peer-cohort models	/backend/ingestion, /backend/models
Person 3	Backend — CUSUM drift, risk fusion, explanation generation, FastAPI layer	/backend/drift, /backend/fusion, /backend/explain, /backend/api

2. Shared Contracts

These two interfaces are the only things all three people need to agree on together, before splitting off to build independently. Agree on these in the first working session; do not revise them mid-build without a quick group check-in, since both sides of each contract depend on it staying stable.

2.1 Contract A — `daily_user_scores` (Person 2 → Person 3)

Person 2 publishes this table (CSV or a lightweight SQLite table both work for a hackathon); Person 3 only ever reads it.

Field	Type	Description
<code>user_id</code>	string	Unique employee identifier
<code>date</code>	date	Day this row's scores apply to
<code>self_score</code>	float 0–1	Isolation Forest anomaly score vs. the user's own history
<code>peer_score</code>	float 0–1	Z-score deviation vs. peer cohort
<code>cohort_used</code>	"role" "department"	Which cohort the peer score was computed against
<code>cohort_size</code>	int	Number of peers in the cohort actually used
<code>fallback_applied</code>	bool	True if department fallback was triggered
<code>role</code>	string	User's current role (context feature)
<code>team</code>	string	User's current team (context feature)
<code>manager_id</code>	string	User's current manager (context feature)

2.2 Contract B — REST API (Person 3 → Person 1)

Person 3 builds and owns these three endpoints; Person 1 only ever calls them, starting with a mocked version matching this exact shape.

GET /queue

Returns the ranked list for the dashboard's main view.

```
[{ "user_id": "u123", "name": "...", "role": "...", "risk_score": 78, "severity": "high", "last_updated": "2026-09-12" }, ...]
```

GET /case/{user_id}

Returns everything the case detail view needs — score breakdown, explanation, and forensic detail.

```
{ "user_id": "u123", "risk_score": 78, "self_score": 0.81, "peer_score": 0.74,
  "drift_score": 0.69, "cohort_used": "department", "fallback_applied": true,
  "explanation_text": "...", "cusum_path": [0.1, 0.2, 0.4, ...],
  "event_timeline": [{ "timestamp": "...", "event": "..." }, ...] }
```

POST /feedback

Records an investigator's false-positive decision, which the fusion engine uses to down-weight that signal combination.

```
{ "user_id": "u123", "is_false_positive": true, "reason": "..." } → 200 OK
```

3. Person 1 — Frontend — Complete Specification

Owens: /frontend (entire folder, sole owner)

Builds the investigator-facing dashboard — the only part of the system any human directly interacts with. Never touches backend code; consumes Contract B only.

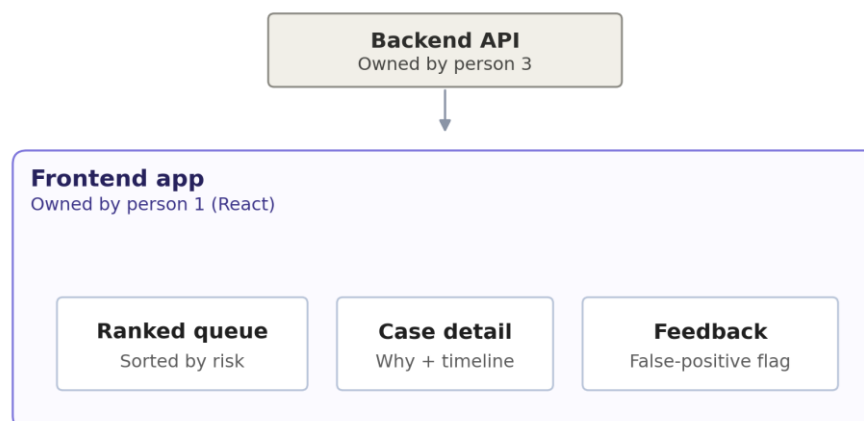
Responsibilities

- **Ranked queue view** — the landing screen — every flagged user sorted by risk_score, with a visible severity tier (medium / high / critical) per Section 9 of the main specification.
- **Case detail view** — on selecting a user: the "why this account now" panel (rendered directly from explanation_text), a forensic drill-down timeline (event_timeline), and a chart of the raw CUSUM path (cusum_path) so an investigator can see the gradual accumulation, not just a pass/fail flag.
- **Feedback control** — a single "mark as false positive" action on every case, posting to /feedback.
- **Mocked-first development** — build against static JSON matching Contract B exactly from hour 0; swap in the real API base URL once Person 3's endpoints are live — no other code changes needed if the contract was followed.

Tech stack

Layer	Technology
Framework	React
Charts	Recharts (for the CUSUM path chart)
Data fetching	fetch / axios against Contract B

Architecture



Deliverables checklist

- ☐ Ranked queue view, sorted and severity-tiered
- ☐ Case detail view with explanation panel
- ☐ Forensic timeline + CUSUM path chart

- ☐ Feedback control wired to POST /feedback
- ☐ Swapped from mocked JSON to Person 3's live API

4. Person 2 — Backend: Data & Detection — Complete Specification

Owens: /backend/ingestion, /backend/models (sole owner)

Owens everything upstream of the shared scores table. Never touches /backend/drift, /backend/fusion, /backend/explain, /backend/api, or /frontend.

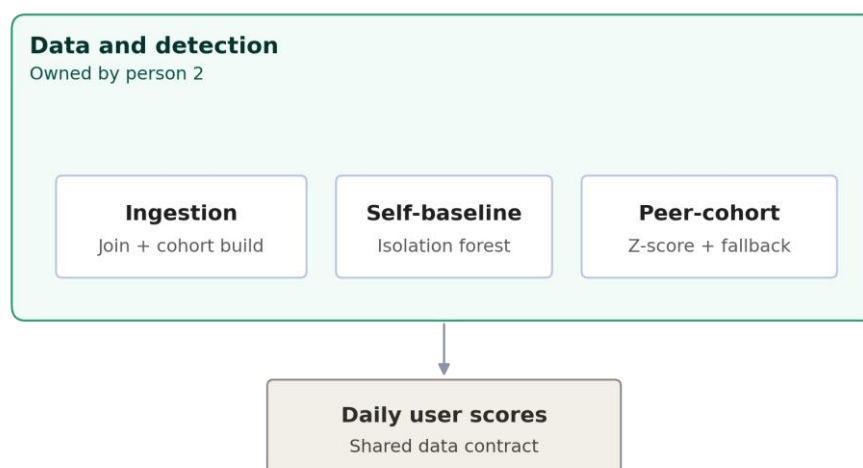
Responsibilities

- **Ingestion** — load CERT r4.2's logon, file, email, http, and device logs; join them per user-day into one working table.
- **Cohort construction** — build role, manager, and department cohort tables from the LDAP file; implement the small-cohort fallback rule (below a minimum cohort size, fall back to department-level) and record fallback_applied + cohort_used per user-day.
- **Self-baseline model** — Isolation Forest trained per user on daily aggregated activity features (logon times, file-access counts, email volume, USB events) — outputs self_score.
- **Peer-cohort model** — z-score of each user's activity against their cohort's mean and spread — outputs peer_score.
- **Publish the contract** — write daily_user_scores (Contract A) on a schedule both backend people agree on early — even an hourly re-write during the hackathon is fine.

Tech stack

Layer	Technology
Data processing	Python, Pandas / Polars
Modeling	scikit-learn (Isolation Forest)
Output	CSV or SQLite table matching Contract A

Architecture



Deliverables checklist

- ☐ CERT logs ingested and joined per user-day
- ☐ Role / manager / department cohort tables built
- ☐ Small-cohort fallback implemented and flagged in output

- ☐ Isolation Forest self-baseline producing self_score
- ☐ Peer z-score model producing peer_score
- ☐ daily_user_scores published matching Contract A exactly

5. Person 3 — Backend: Fusion, Drift & API — Complete Specification

Owens: `/backend/drift`, `/backend/fusion`, `/backend/explain`, `/backend/api` (sole owner)

Owens everything downstream of the shared scores table. Reads Contract A; never edits Person 2's folders. Publishes Contract B; never edits `/frontend`.

Responsibilities

- **CUSUM drift detection** — runs on the pre-multiplier fused daily score — the simple combination of `self_score` and `peer_score`, before any multiplier is applied. This exact input is locked so the drift signal stays stable regardless of later multiplier tuning. Store the raw cumulative-sum path per user for the frontend's forensic chart.
- **Risk fusion engine** — combines `self_score`, `peer_score`, and the CUSUM drift signal into one 0–100 `risk_score`, adjusted by the role/time/data-sensitivity multipliers.

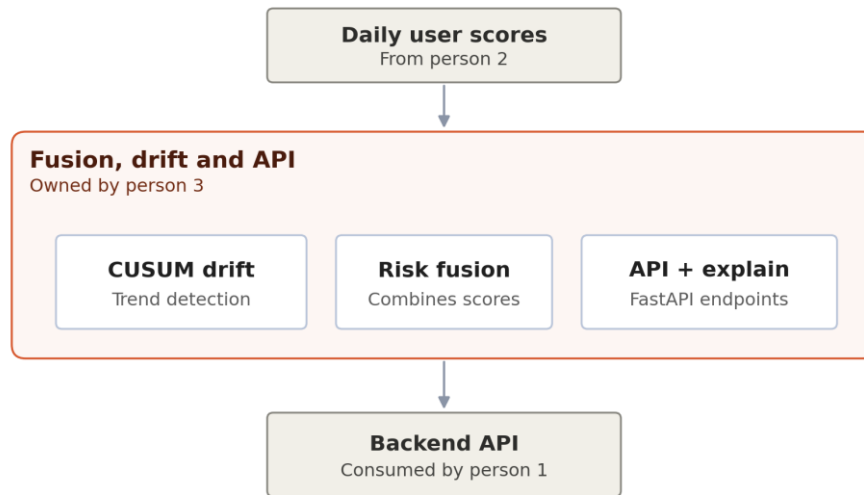
Multiplier	Condition	Effect
Role sensitivity	Admin / privileged account	Higher weight
Time of access	Before 7am or after 8pm	Higher weight
Data sensitivity	Confidential-tagged resource, removable media	Higher weight
Score decay	Each day without new anomalous activity	Gradual weight reduction

- **Explanation generator** — templated natural-language text composed directly from `self_score`, `peer_score`, drift, and the fallback flag — e.g. stating explicitly when a department-level fallback was used, per Section 7.2 of the main specification.
- **FastAPI layer** — implements the three Contract B endpoints; persists feedback from `/feedback` and applies the down-weighting logic for that signal combination going forward.

Tech stack

Layer	Technology
Drift & fusion	NumPy / SciPy (CUSUM), Python
API	FastAPI
Feedback persistence	SQLite or in-memory store, hackathon-scale

Architecture



Deliverables checklist

- ☐ CUSUM running on the locked pre-multiplier score; raw path stored
- ☐ Risk fusion engine with all four multipliers
- ☐ Explanation generator, including fallback disclosure
- ☐ GET /queue, GET /case/{id}, POST /feedback all live and matching Contract B
- ☐ Feedback down-weighting implemented

6. Master Build Timeline

Hour	Person 1 (Frontend)	Person 2 (Data & Detection)	Person 3 (Fusion, Drift & API)
0–1	Agree Contracts A & B together	Agree Contracts A & B together	Agree Contracts A & B together
1–8	Build queue + case views vs. mocked JSON	Ingest CERT logs, build cohort tables	Build FastAPI skeleton vs. fake scores CSV
8–20	Build drill-down timeline + CUSUM chart (mocked)	Isolation Forest + peer z-score + fallback; publish real table	Continue vs. fake data — not blocked by Person 2
20–28	Polish explanation panel	Support integration, refine cohort edge cases	Swap to Person 2's real table; wire CUSUM
28–36	Wire feedback button (mocked)	Free to help, or start LSTM stretch	Build risk fusion + explanation generator
36–44	Swap to Person 3's live API	—	Finalize API, support integration
44–48	Final polish, rehearse demo	Support demo	Support demo, feedback loop polish

7. Git & Merge-Conflict Prevention

Folder separation removes most conflict risk by itself. Two habits make it airtight:

- **One branch per person** — frontend, backend-detection, backend-fusion — each person commits freely on their own branch without coordinating.
- **Merge to main only at the three sync points** — hour 20 (Person 2's table goes live), hour 36 (Person 3's API goes live), and hour 44 (final polish merge). Outside those moments, branches don't need to touch main at all.

Because no two people ever edit the same file, a merge at any sync point is a fast-forward or a trivial merge — there is structurally nothing to resolve.

8. Sync Points & Handoff Checklist

Sync point 1 — hour 0–1: Contracts agreed

- ☐ Contract A (daily_user_scores) fields finalized
- ☐ Contract B (API shape) finalized
- ☐ Everyone has a mocked/fake version of what they consume

Sync point 2 — hour ~20: Real scores table live

- ☐ Person 2 publishes daily_user_scores matching Contract A exactly
- ☐ Person 3 swaps fake CSV for the real table

Sync point 3 — hour ~36: Real API live

- ☐ Person 3's three endpoints match Contract B exactly
- ☐ Person 1 swaps mocked JSON for the live API base URL

Sync point 4 — hour ~44: Final merge

- ☐ All three branches merged to main
- ☐ Full end-to-end run rehearsed at least once before the demo